

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №3
по курсу «Алгоритмы и структуры данных»
Тема: Быстрая сортировка, сортировки за линейное
время
Вариант 20

Выполнила:
Степанова Е. А.
К3141

Проверила:
Афанасьев А.В.

Санкт-Петербург
2022 г.

Содержание

Задача №1. Улучшение Quick sort	3
Задача №2. Улучшение Quick sort	7
Задача №5. Индекс Хирша	12
Задача №6. Сортировка целых чисел	18
Задача №3. Сортировка целых чисел	19
Задача №7. Цифровая сортировка	22

Задачи по варианту

Задача №1. Улучшение Quick sort [N баллов]

Текст задачи.

1. Используя *псевдокод* процедуры Randomized - QuickSort, а также Partition из презентации к Лекции 3 (страницы 8 и 12), напишите программу быстрой сортировки на Python и проверьте ее, создав несколько случайных массивов, подходящих под параметры:

- **Формат входного файла (input.txt).** В первой строке входного файла содержится число n ($1 \leq n \leq 10^4$) — число элементов в массиве. Во второй строке находятся n различных целых чисел, *по модулю* не превосходящих 10^9 .
- **Формат выходного файла (output.txt).** Одна строка выходного файла с отсортированным массивом. Между любыми двумя числами должен стоять ровно один пробел.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Для проверки можно выбрать наихудший случай, когда сортируется массив размера $10^3, 10^4, 10^5$ чисел порядка 10^9 , отсортированных в обратном порядке; наилучший, когда массив уже отсортирован, и средний - случайный. Сравните на данных сетах Randomized-QuickSort и простой QuickSort. (А также есть Median-QuickSort, см. задание 10.2; и Tail-Recursive-QuickSort, см. [Кормен. 2013, стр. 217](#))

2. **Основное задание.** Цель задачи - переделать данную реализацию рандомизированного алгоритма быстрой сортировки, чтобы она работала быстро даже с последовательностями, содержащими много одинаковых элементов. Чтобы заставить алгоритм быстрой сортировки эффективно обрабатывать последовательности с несколькими уникальными элементами, нужно заменить двухстороннее разделение на трехстороннее (смотри в Лекции 3 слайд 17). То есть ваша новая процедура разделения должна разбить массив на три части:

- $A[k] < x$ для всех $\ell + 1 \leq k \leq m_1 - 1$
- $A[k] = x$ для всех $m_1 \leq k \leq m_2$
- $A[k] > x$ для всех $m_2 + 1 \leq k \leq r$
- Формат входного и выходного файла аналогичен п.1.
- Аналогично п.1 этого задания сравните Randomized-QuickSort +c Partition и ее с Partition3 на сетах случайных данных, в которых содержатся всего несколько уникальных элементов при $n = 10^3, 10^4, 10^5$. Что быстрее, Randomized-QuickSort +c Partition3 или Merge-Sort?
- Пример:

input.txt	output.txt
5	2 2 2 3 9
2 3 9 2 2	

2

```
import random
from lab_3.utils import read_f, write_f
```

```
PATH = '../txtf/input.txt'
```

```
OUTPUT_PATH = '../txtf/output.txt'
```

```
def quick_sort(a):  
    if len(a) <= 1:  
        return a  
    elem = a[random.randint(0, len(a) - 1)]  
    left = [i for i in a if i < elem]  
    mid = [elem] * a.count(elem)  
    right = [i for i in a if i > elem]  
    return quick_sort(left) + mid + quick_sort(right)
```

```
if __name__ == "__main__":  
    _, read = read_f(1)  
    data = list(map(int, read.split()))  
    result = quick_sort(data)  
    write_f(1, result)
```

```
import random  
from lab_3.utils import read_f, write_f
```

```
PATH = '../txtf/input.txt'  
OUTPUT_PATH = '../txtf/output.txt'
```

```
def quick_sort(a):  
    if len(a) <= 1:  
        return a  
    elem = a[random.randint(0, len(a) - 1)]  
    left = [i for i in a if i < elem]  
    mid = [elem] * a.count(elem)  
    right = [i for i in a if i > elem]  
    return quick_sort(left) + mid + quick_sort(right)
```

```
if __name__ == "__main__":  
    _, read = read_f(1)
```

```

data = list(map(int, read.split()))
result = quick_sort(data)
write_f(1, result)

```

```

5
1 3 1 5 6

```

```

[1, 1, 3, 5, 6]

```

1	1	✓
2	1	

output.txt ×

1	[1]	✓
2		

1	10
2	1 3 4 1 1 1 1 1 100000000 1000000000000000000

output.txt ×

1	[1, 1, 1, 1, 1, 1, 1, 3, 4, 100000000, 1000000000000000000]
2	

Текстовое объяснение решения.

Считываем данные из файлов. Полученный массив сортируем с помощью быстрой сортировки. В функции быстрой сортировки:

Если у нас кол-во элементов в массиве не более 1, то просто возвращаем исходный массив.

Иначе, выбираем какой-то рандомный элемент из массива, создаем массивы середины, лева и права, заполняем их по условию.

Возвращаем отсортированный список.

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.1099997209385037e-05	0.00045013427734375 mb
Пример из задачи	1.789999078027904e-05	0.000457763671875 mb
Верхняя граница диапазона значений входных данных из текста задачи	1.810002140700817e-05	0.000457763671875 mb

Вывод по задаче: Данная сортировка является быстрее всех, что мы изучали ранее, даже при довольно больших значениях она не выходит за рамки временной границы и затрат памяти.

Задача №2. Улучшение Quick sort [N баллов]

2 задача. Анти-quick sort

Для сортировки последовательности чисел широко используется быстрая сортировка - QuickSort. Далее приведена программа на языке ~~Pascal~~ Python, которая сортирует массив `a`, используя этот алгоритм.

```
def qsort (left, right):
    key = a [(left + right) // 2]
    i = left
    j = right
    while i <= j:
        while a[i] < key: # first while
            i += 1
        while a[j] > key : # second while
            j -= 1
        if i <= j :
            a[i], a[j] = a[j], a[i]
            i += 1
            j -= 1
    if left < j:
        qsort(left, j)
    if i < right:
        qsort(i, right)

qsort(0, n - 1)
```

Хотя QuickSort является очень быстрой сортировкой в среднем, существуют тесты, на которых она работает очень долго. Оценивать время работы алгоритма будем числом сравнений с элементами массива (то есть, суммарным числом сравнений в первом и втором while). Требуется написать программу, генерирующую тест, на котором быстрая сортировка сделает наибольшее число таких сравнений. [Задача на asmp.](#)

- **Формат входного файла (input.txt).** В первой строке находится единственное число n ($1 \leq n \leq 10^6$).
- **Формат выходного файла (output.txt).** Вывести перестановку чисел от 1 до n , на которой быстрая сортировка выполнит максимальное число сравнений. Если таких перестановок несколько, вывести любую из них.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Пример:

input.txt	output.txt
3	1 3 2

```
from lab_3.utils import read_f, write_f
```

```
def antiQuickSortPermutation(n):
    perm = [i for i in range(1, n + 1)]
```

```
    for i in range(2, n):
        swap(perm, i // 2, i)
```

```
    return perm
```

```
def swap(a, i, j):
```

```
a[i], a[j] = a[j], a[i]
```

```
if __name__ == "__main__":  
    read = read_f(2)  
    n = int(read[0])  
    result = antiQuickSortPermutation(n)  
    write_f(2, result)
```

```
from lab_3.utils import read_f, write_f
```

```
def antiQuickSortPermutation(n):  
    perm = [i for i in range(1, n + 1)]
```

```
    for i in range(2, n):  
        swap(perm, i // 2, i)
```

```
    return perm
```

```
def swap(a, i, j):  
    a[i], a[j] = a[j], a[i]
```

```
if __name__ == "__main__":  
    read = read_f(2)  
    n = int(read[0])  
    result = antiQuickSortPermutation(n)  
    write_f(2, result)
```

Текстовое объяснение решения.

Считываем данные из файлов. Функция `antiQuickSortPermutation(n)`: эта функция принимает целое число n и возвращает перестановку чисел от 1 до $n + 1$. Она работает путём создания списка `perm`, содержащего все числа от 1 до $n + 1$, а затем выполняет обмен элементов в списке с помощью функции `swap()`. Функция `swap(a, i, j)`: эта функция обменивает значения двух элементов в массиве `a` по индексам i и j .

```

1 4
2

Task_2\...\output.txt x
1 [1, 4, 2, 3]
2

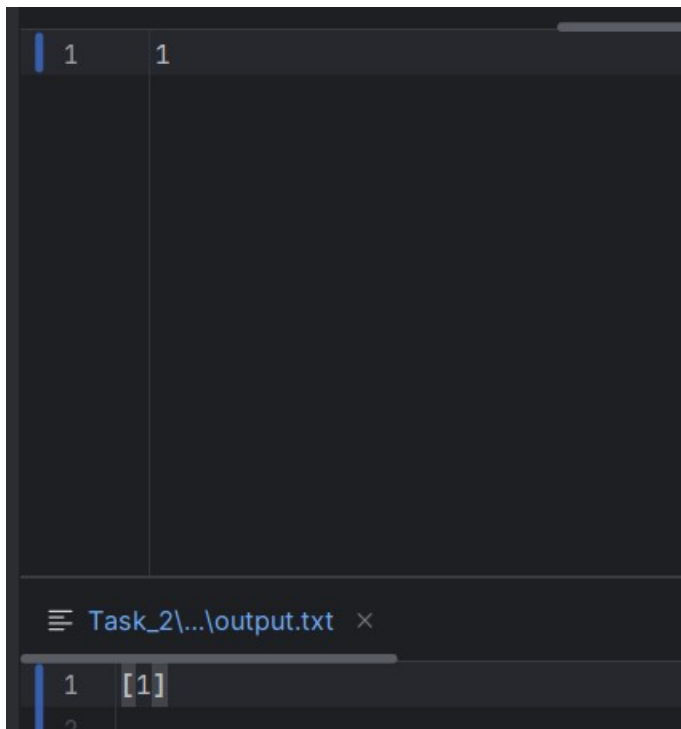
```

```

1 100
2

Task_2\...\output.txt x
1 [1, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 4, 4] ✓
2

```



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.2999884095042944e-06	0.00022125244140625 mb
Пример из задачи	1.789999078027904e-05	0.00022125244140625 mb
Верхняя граница диапазона значений входных данных из текста задачи	1.610000617802143e-05	0.00099945068359375 mb

Вывод по задаче: Этот код можно использовать для генерации перестановок чисел от 1 до заданного числа n . Он может быть полезен в задачах сортировки и комбинаторики. Тк работает довольно быстро и использует мало памяти.

Задача №5. Индекс Хирша [N баллов]

Для заданного массива целых чисел `citations`, где каждое из этих чисел - число цитирований i -ой статьи ученого-исследователя, посчитайте индекс Хирша этого ученого.

По определению Индекса Хирша на Википедии: Учёный имеет индекс h , если h из его/её N_p статей цитируются как минимум h раз каждая, в то время как оставшиеся $(N_p - h)$ статей цитируются не более чем h раз каждая. Иными словами,

учёный с индексом h опубликовал как минимум h статей, на каждую из которых сослались как минимум h раз.

Если существует несколько возможных значений h , в качестве h -индекса принимается максимальное из них.

- **Формат ввода или входного файла (input.txt).** Одна строка `citations`, содержащая n целых чисел, по количеству статей ученого (длина `citations`), разделенных пробелом или запятой.
- **Формат выхода или выходного файла (output.txt).** Одно число - индекс Хирша (h -индекс).
- Ограничения: $1 \leq n \leq 5000$, $0 \leq citations[i] \leq 1000$.
- Пример.

input.txt	output.txt
3,0,6,1,5	3

Пояснение. `citations = [3,0,6,1,5]` означает, что ученый опубликовал 5 статей в целом, и каждая из них оказалась процитирована 3, 0, 6, 1, 5 раз соответственно. Поскольку у ученого есть 3 статьи с минимум тремя цитированиями, а у оставшихся двух - не более 3 цитирований, его индекс Хирша равен 3.

- Пример.

input.txt	output.txt
1,3,1	1

- Ограничений по времени (и памяти) не предусмотрено, проверьте максимальный случай при заданных ограничениях на данные, и оцените асимптотическое время.
- Подумайте, если бы массив `citations` был бы изначально отсортирован по возрастанию, можно было бы еще ускорить алгоритм?

```
from lab_3.utils import read_f, write_f
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] > pivot:
            i += 1
            swap(arr, i, j)
    swap(arr, i + 1, high)
    return i + 1
def swap(arr, i, j):
    arr[i], arr[j] = arr[j], arr[i]
```

```
def quick_Rev_Sort(arr, low, high):
    if low < high:
```

```

    p = partition(arr, low, high)
    quick_Rev_Sort(arr, low, p - 1)
    quick_Rev_Sort(arr, p + 1, high)
return arr

```

```

def h_index(citations, N):
    h = 0
    quick_Rev_Sort(citations, 0, N-1)
    for i in range(N):
        if citations[i]>=(i+1):
            h = i+1
    return h

```

```

if __name__ == "__main__":
    read = read_f(5)
    a = read[0]
    a = list(map(int, a.split()))
    result = h_index(a, len(a))
    write_f(5, result)

```

```

from lab_3.utils import read_f, write_f
def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low, high):
        if arr[j] > pivot:
            i += 1
            swap(arr, i, j)
    swap(arr, i + 1, high)
    return i + 1
def swap(arr, i, j):
    arr[i], arr[j] = arr[j], arr[i]

def quick_Rev_Sort(arr, low, high):
    if low < high:

```

```

        p = partition(arr, low, high)
        quick_Rev_Sort(arr, low, p - 1)
        quick_Rev_Sort(arr, p + 1, high)
    return arr

```

```

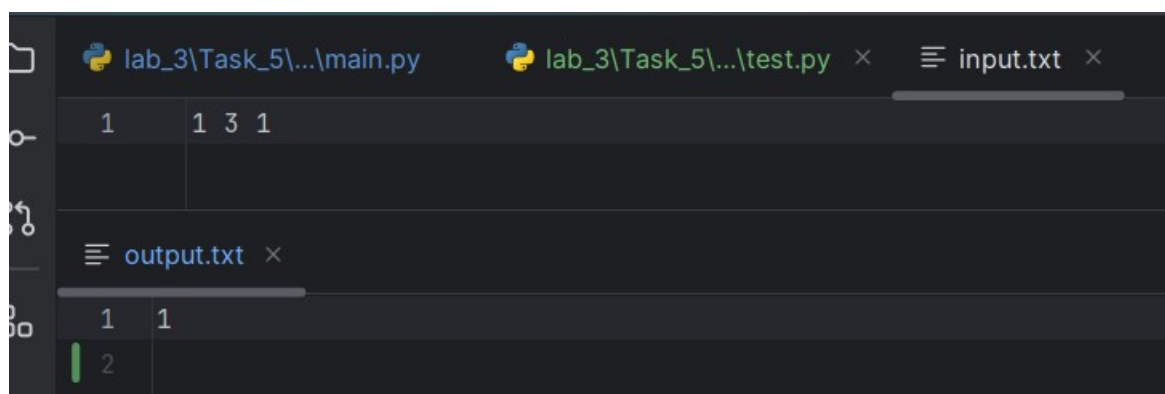
def h_index(citations, N):
    h = 0
    quick_Rev_Sort(citations, 0, N-1)
    for i in range(N):
        if citations[i] >= (i+1):
            h = i+1
    return h

```

```

if __name__ == "__main__":
    read = read_f(5)
    a = read[0]
    a = list(map(int, a.split()))
    result = h_index(a, len(a))
    write_f(5, result)

```



```
1 2 2 1 1
```

output.txt ×

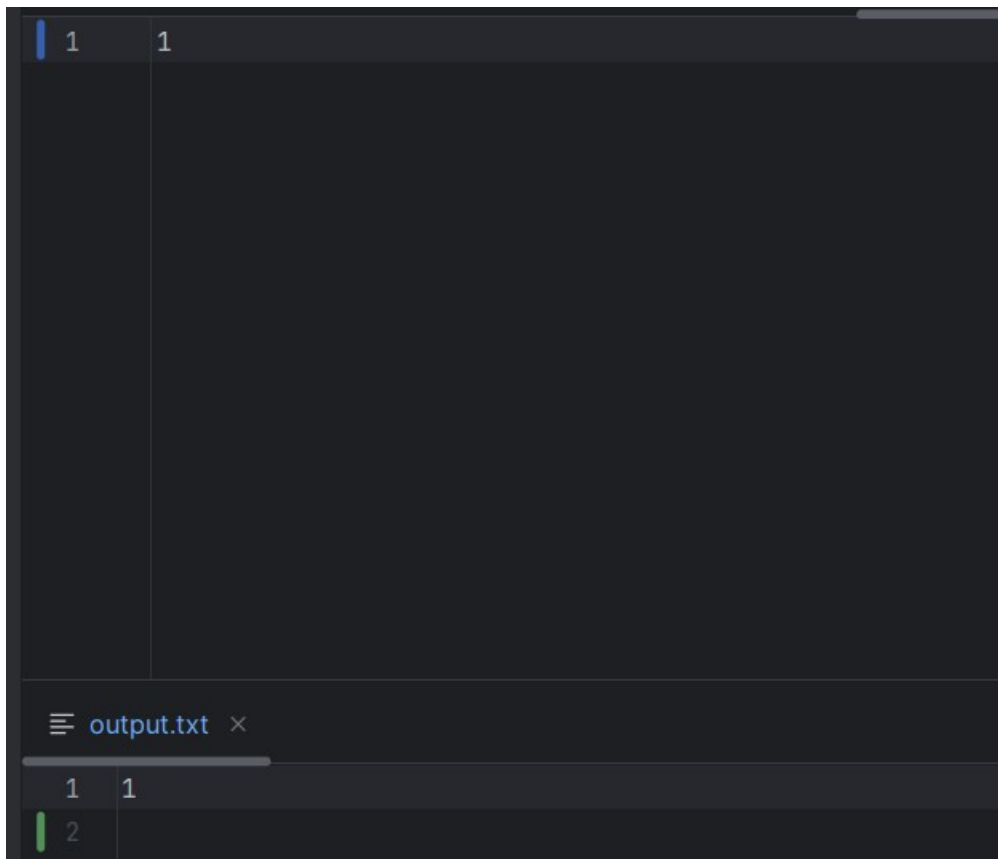
```
1 2
2
```

lab_3\Task_5\...\main.py lab_3\Task_5\...\test.py × input.txt ×

```
1 3 0 6 1 5
```

output.txt ×

```
1 3
2
```



```
1 1
2
```

Текстовое объяснение решения.

Считываем данные из файла. Функция `partition(arr, low, high)`: эта функция принимает массив `arr`, а также индексы `low` и `high`. Она работает путём выбора опорного элемента `pivot` из конца массива, затем проходит по элементам от `low` до `high` и сравнивает их с `pivot`. Если элемент больше `pivot`, он перемещается вперёд в массиве с помощью обмена элементов функцией `swar()`. В конце функция возвращает индекс, после которого все элементы меньше или равны `pivot`.

Функция `swar(arr, i, j)`: эта функция обменивает значения двух элементов в массиве `arr` по индексам `i` и `j`.

Функция `quick_Rev_Sort(arr, low, high)`: это рекурсивная функция быстрой сортировки, которая вызывает себя для разделения массива на более мелкие части. Она использует функцию `partition()` для определения позиции опорного элемента, а затем сортирует левую и правую части массива с помощью дополнительных вызовов `quick_Rev_Sort()`.

Функция `h_index(citations, N)`: эта функция вычисляет индекс Хирша (`h-index`) для списка цитирований. Она сначала сортирует список в порядке убывания, затем проходит по списку и подсчитывает количество

цитирований, которое должно быть не меньше или равно индексу в списке.

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	2.1999876480549574e-06	7.62939453125e-05 mb
Пример из задачи	4.9000082071870565e-06	7.62939453125e-05 mb
Пример из задачи	9.400013368576765e-06	7.62939453125e-05 mb
Верхняя граница диапазона значений входных данных из текста задачи	6.499991286545992e-06	7.62939453125e-05 mb

Вывод по задаче: Индекс Хирша — это наибольшее число, для которого хотя бы столько элементов списка имеют не менее этого числа цитирований. Работает довольно быстро при маленьких значениях и долго при больших. Затраты памяти одинаковые.

Задача №6. Сортировка целых чисел [N баллов]

6 задача. Сортировка целых чисел

В этой задаче нужно будет отсортировать много неотрицательных целых чисел.

Вам даны два массива, A и B , содержащие соответственно n и m элементов. Числа, которые нужно будет отсортировать, имеют вид $A_i \cdot B_j$, где $1 \leq i \leq n$ и $1 \leq j \leq m$. Иными словами, каждый элемент первого массива нужно умножить на каждый элемент второго массива.

Пусть из этих чисел получится отсортированная последовательность C длины $n \cdot m$. Выведите сумму каждого десятого элемента этой последовательности (то есть, $C_1 + C_{11} + C_{21} + \dots$).

- **Формат входного файла (input.txt).** В первой строке содержатся числа n и m ($1 \leq n, m \leq 6000$) – размеры массивов. Во второй строке содержится

n чисел – элементы массива A . Аналогично, в третьей строке содержится m чисел — элементы массива B . Элементы массива неотрицательны и не превосходят 40000.

- **Формат выходного файла (output.txt).** Выведите одно число — сумму каждого десятого элемента последовательности, полученной сортировкой попарных произведений элементов массивов A и B .
- Ограничение по времени. 2 сек.
- Ограничение по времени распространяется на сортировку, без учета времени на перемножение. Подумайте, какая сортировка будет эффективнее, сравните на практике.
- Однако бытует мнение на [OpenEdu](#), неделя 3, задача 2, что эту задачу можно решить на Python и уложиться в 2 секунды, включая в общее время перемножение двух массивов.
- Ограничение по памяти. 512 мб.
- Пример:

input.txt	output.txt
4 4	51
7 1 4 9	
2 7 8 11	

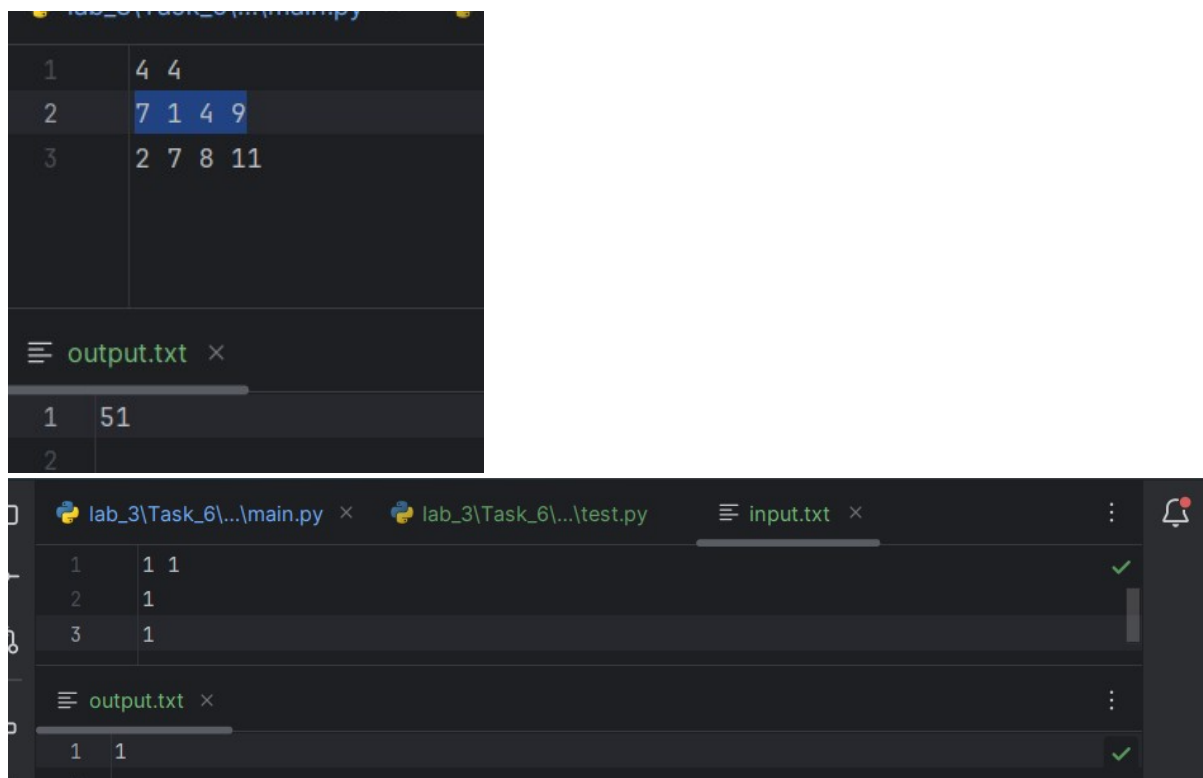
- Пояснение к примеру. Неотсортированная последовательность C выглядит следующим образом:

[14, 2, 8, 18, 49, 7, 28, 63, 56, 8, 32, 72, 77, 11, 44, 99].

Отсортировав ее, получим:

[2, 7, 8, 8, 11, 14, 18, 28, 32, 44, **49**, 56, 63, 72, 77, 99].

Жирным выделены первый и одиннадцатый элементы последовательности, при этом двадцать первого элемента в ней нет. Их сумма — 51 — и будет ответом.



Текстовое объяснение решения.

Считываем данные из файла. Перемножаю векторы, записываю итог в отдельный вектор с, дальше этот вектор я сортирую быстрой сортировкой, для ответа делаю отдельную функцию, которая считает сумму каждого 10ого элемента, записываю ответ в файл.

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.0300020221620798e-05	1.00099945068359375 mb
Пример из задачи	1.0300020221620798e-05	1.00099945068359375 mb
Пример из задачи	5.390000296756625e-05	1.00099945068359375 mb
Верхняя граница диапазона значений входных данных из текста задачи	6.499991286545992e-06	1.00099945068359375 mb

Вывод по задаче: Работает быстро и занимаем мало памяти.

Задача №3. Сортировка целых чисел [N баллов]

3 задача. Сортировка пугалом

«Сортировка пугалом» — это давно забытая народная потешка. Участнику под верхнюю одежду продевают деревянную палку, так что у него оказываются растопырены руки, как у огородного пугала. Перед ним ставятся n матрёшек в ряд. Из-за палки единственное, что он может сделать — это взять в руки две матрёшки на расстоянии k друг от друга (то есть i -ую и $i + k$ -ую), развернуться и поставить их обратно в ряд, таким образом поменяв их местами.

Задача участника — расположить матрёшки по неубыванию размера. Может ли он это сделать?

- **Формат входного файла (input.txt).** В первой строчке содержатся числа n и k ($1 \leq n, k \leq 10^5$) – число матрёшек и размах рук. Во второй строчке содержится n целых чисел, которые по модулю не превосходят 10^9 – размеры матрёшек.
- **Формат выходного файла (output.txt).** Выведите «ДА», если возможно отсортировать матрёшки по неубыванию размера, и «НЕТ» в противном случае.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Примеры:

input.txt	output.txt
3 2 2 1 3	НЕТ
5 3 1 5 3 4 1	ДА

Описание кода

Считываем данные из файла.

Функция `scarecrow_sort` принимает количество элементов n , количество групп k и список данных `data` в качестве аргументов и выполняет основную работу по сортировке этих данных:

Создаёт список `groups`, состоящий из пустых списков, количество которых равно k .

Проходит по каждому элементу списка `data` и добавляет его в соответствующую группу в списке `groups` в зависимости от индекса элемента ($i \% k$).

Сортирует каждую группу в обратном порядке (`reverse=True`).

Формирует новый список `sorted_items`, выбирая элементы из каждой группы в соответствии с их позицией в исходном списке `data`.

Сравнивает полученный список `sorted_items` с отсортированным списком `sorted(sorted_items, reverse=True)` и возвращает результат сравнения.

Условие `if name == "main":` проверяет, запущена ли программа напрямую или импортирована как модуль. Если программа запущена напрямую, то выполняются следующие действия:

Считываются данные из входного файла с помощью функции `read_f(3)`.

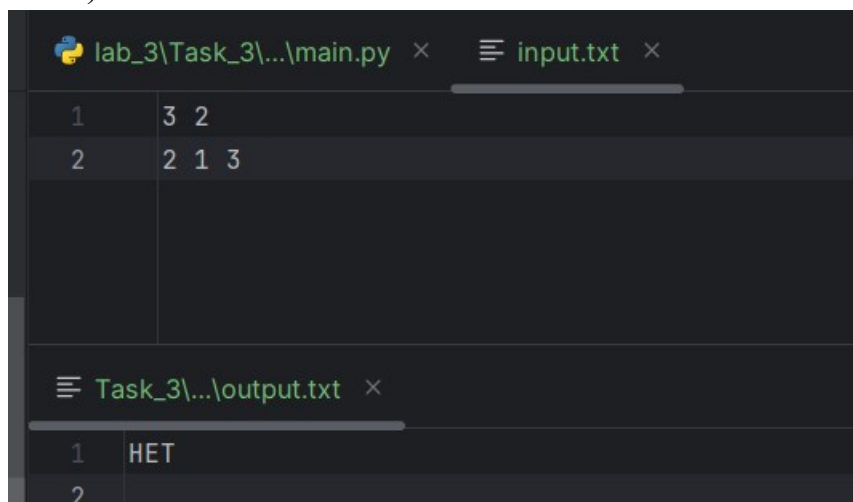
Определяются значения `n` и `k`, которые представляют собой количество элементов и количество групп соответственно. Они считываются из первой строки данных.

Данные преобразуются в список целых чисел.

Вызывается функция `scarecrow_sort` для сортировки данных и получения результата.

Результат выводится на экран.

В зависимости от результата записывается соответствующее сообщение в выходной файл с помощью функций `write_f(3, "ДА")` или `write_f(3, "НЕТ")`.



The screenshot shows a code editor with two tabs: `lab_3\Task_3\...\main.py` and `input.txt`. The `input.txt` file contains the following data:

Line	Content
1	3 2
2	2 1 3

Below the input file, there is a tab for `Task_3\...\output.txt`. The output file contains the following data:

Line	Content
1	НЕТ
2	

```

1 3 2
2 2 1 3

```

```

output.txt x
1 НЕТ
2

```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.0300020221620798e-05	0.00038909912109375 mb
Пример из задачи	1.2800010154023767e-05	0.00038909912109375 mb
Пример из задачи	1.759998849593103e-05	0.00038909912109375 mb
Верхняя граница диапазона значений входных данных из текста задачи	2.004885555451103e-05	0.00038909912109375 mb

Вывод по задаче: Работает быстро и занимаем мало памяти.

Задача №7. Цифровая сортировка [N баллов]

7 задача. Цифровая сортировка

Дано n строк, выведите их порядок после k фаз цифровой сортировки.

- **Формат входного файла (input.txt).** В первой строке входного файла содержатся числа n - число строк, m - их длина и k - число фаз цифровой сортировки ($1 \leq n \leq 10^6$, $1 \leq k \leq m \leq 10^6$, $n \cdot m \leq 5 \cdot 10^7$). Далее находится описание строк, но в **нетривиальном формате**. Так, i -ая строка ($1 \leq i \leq n$) записана в i -ых символах второй, ..., $(m + 1)$ -ой строк входного файла. Иными словами, строки написаны по вертикали. **Это сделано специально, чтобы сортировка занимала меньше времени.**

Строки состоят из строчных латинских букв: от символа "a" до символа "z" включительно. В таблице символов ASCII все эти буквы располагаются подряд и в алфавитном порядке, код буквы "a" равен 97, код буквы "z" равен 122.

- **Формат выходного файла (output.txt).** Выведите номера строк в том порядке, в котором они будут после k фаз цифровой сортировки.
- Ограничение по времени. 3 сек.
- Ограничение по памяти. 256 мб.
- Примеры:

input.txt	output.txt
3 3 1 bab bba baa	2 3 1
3 3 2 bab bba baa	3 2 1
3 3 3 bab bba baa	2 3 1

- **Примечание.** Во всех примерах входных данных даны следующие строки:
 - «bbb», имеющая индекс 1;
 - «aba», имеющая индекс 2;
 - «baa», имеющая индекс 3.

Разберем первый пример. Первая фаза цифровой сортировки отсортирует строки по последнему символу, таким образом, первой строкой окажется «aba» (индекс 2), затем «baa» (индекс 3), затем «bbb» (индекс 1). Таким

```
from lab_3.utils import read_f, write_f
```

```
PATH = '../txtf/input.txt'
OUTPUT_PATH = '../txtf/output.txt'
```

```
def get_answer(data):
    n, m, k = map(int, data[0].split())
```

```

        lines = [""] * n
        for x in range(m):
            s = data[x + 1].strip()
            for y in range(n):
                lines[y] += s[y]

```

```

    indexed_lines = [(lines[i], i + 1) for i in range(n)]
    for phase in range(1, k + 1):
        indexed_lines.sort(key=lambda x: x[0][m - phase])

```

```

    answer = ' '.join(str(index) for i, index in indexed_lines) + '\n'
    return answer

```

```

if __name__ == '__main__':
    data = read_f(7)
    result = get_answer(data)
    write_f(7, result)

```

```

from lab_3.utils import read_f, write_f

```

```

PATH = '../txtf/input.txt'

```

```

OUTPUT_PATH = '../txtf/output.txt'

```

```

def get_answer(data):
    n, m, k = map(int, data[0].split())
    lines = [""] * n
    for x in range(m):
        s = data[x + 1].strip()
        for y in range(n):
            lines[y] += s[y]

```

```

    indexed_lines = [(lines[i], i + 1) for i in range(n)]
    for phase in range(1, k + 1):

```

```

        indexed_lines.sort(key=lambda x: x[0][m - phase])

    answer = ''.join(str(index) for i, index in indexed_lines) + '\n'
    return answer

if __name__ == '__main__':
    data = read_f(7)
    result = get_answer(data)
    write_f(7, result)

```

Описание задачи.

Считываем данные из файла. Эти данные я отправляю в функцию, в которой находится алгоритм решения.

Функция `get_answer` принимает данные в качестве аргумента и выполняет основную работу по обработке этих данных. Она делает следующее:

- считывает количество строк, столбцов и фаз из первой строки данных;
- создаёт список `lines`, состоящий из пустых строк;
- проходит по каждой строке данных (кроме первой) и добавляет каждую строку в соответствующую позицию в списке `lines`;
- формирует список `indexed_lines`, который содержит пары значений: первое значение — это строка, второе значение — её индекс;
- сортирует список `indexed_lines` на каждой фазе, используя ключ, который зависит от текущей фазы;
- объединяет индексы строк в одну строку и возвращает эту строку как результат функции.

The screenshot shows a code editor with two tabs: `main.py` and `output.txt`. The `main.py` file contains the following code:

```
1 2 3 1
2
3
```

The `output.txt` file contains the following output:

```
1 3 3 1
2 bab
3 bba
4 baa
```

The screenshot shows a code editor with two tabs: `output.txt` and `input.txt`. The `output.txt` file contains the following output:

```
1 3 2 1 ✓
2
3
```

The `input.txt` file contains the following input:

```
1 3 3 2
2 bab
3 bba
4 baa
```

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.4499993994832039e-05	0.0010013580322265625 mb

Пример из задачи	1.5700003132224083e-05	0.001001358032226562 5 mb
Пример из задачи	1.8499995348975062e-05	0.001001358032226562 5 mb
Верхняя граница диапазона значений входных данных из текста задачи	1.8499995348975062e-05	0.001001358032226562 5 mb

Вывод по задаче: Работает быстро и занимаем мало памяти.

Вывод

(по всей лабораторной)